

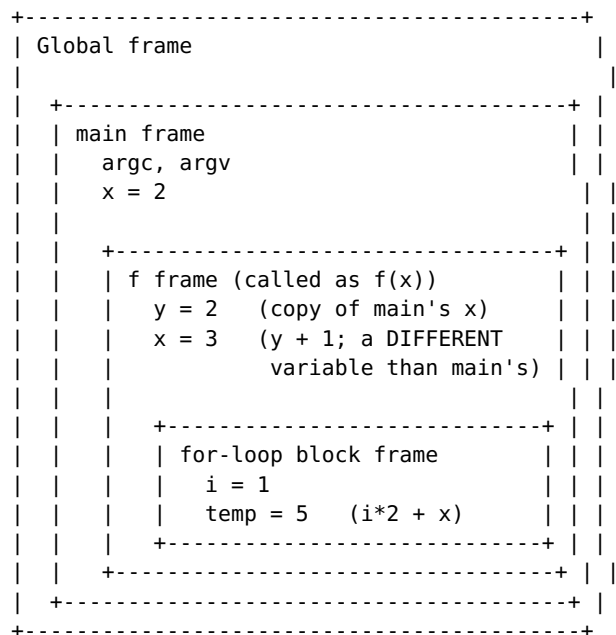
Assignment 05 - Question 3: Scope / Contour Diagrams

Part (a)

```
void f(int y) {
    int x = y + 1;
    for (int i = 0; i < 3; i++) {
        int temp = i*2 + x;
        std::cout << i << " " << temp << " " << x << std::endl;
    }
}

int main(int argc, char *argv[]) {
    int x = 2;
    f(x);
    std::cout << x << std::endl;
    return 0;
}
```

Contours, at the point inside the loop body when `i == 1`:



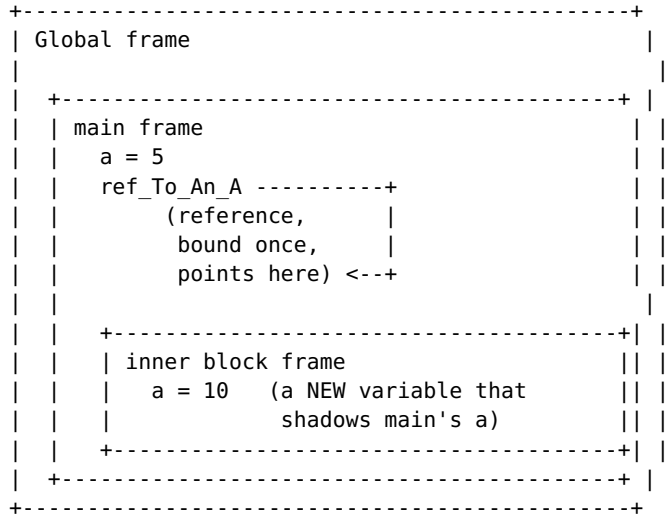
Key point: `f`'s parameter `y` and local `x` live in `f`'s own frame, nested inside `main`'s frame only because `main` called `f` - they are not the same box as `main`'s `x`. Since `y` is passed by value, `f` cannot change `main`'s `x`. The for loop opens one more nested frame for `i` and `temp`, which disappears each time the loop body finishes an iteration. After `f` returns, its frame (and the loop's) are gone, so the final `std::cout << x` in `main` prints 2, unaffected by anything that happened inside `f`.

Part (b)

```
int main() {
    int a = 5;
    int & ref_To_An_A = a;
    {
```

```
int a = 10;
std::cout << ref << std::endl;
}
return 0;
}
```

Note: as written this snippet doesn't compile - the `std::cout` line refers to `ref`, but only `ref_To_An_A` was declared. Treating that as a typo for `ref_To_An_A` (the interesting scope question here is about the reference, not the typo):



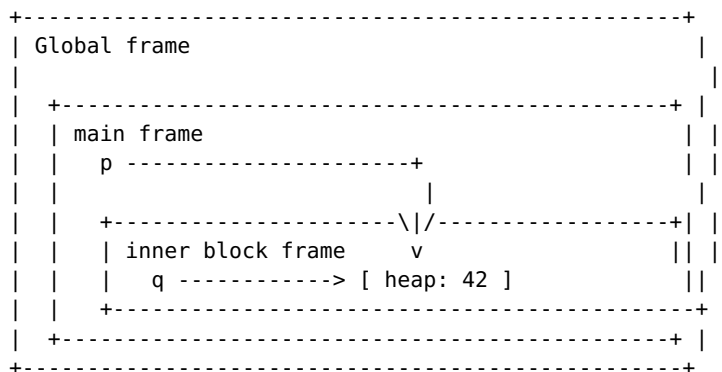
Key point: `ref_To_An_A` is bound to the outer `a` at the moment it's declared, and a reference cannot be re-bound later. The inner block's `int a = 10;` creates a brand-new variable that merely shadows the name `a` for lookups by that name inside the block - it has no effect on `ref_To_An_A`, which still refers to the outer `a` (value 5). So `std::cout << ref_To_An_A` inside the block would print 5, not 10.

Parts (c) and (d)

Both parts give the exact same code:

```
int main() {
    int* p = new int(42);
    {
        int* q = p;
        std::cout << *q << std::endl;
    }
    delete p;
}
```

Contours, at the point inside the inner block:



Key point: `p` and `q` are two different pointer variables (in two different frames), but they hold the same address, so both point at the one `int` object living on the heap, outside of any frame. When the inner block ends, `q` goes out of scope - only the pointer variable is destroyed, not the heap object it pointed to (`p` still points at it, and nothing has been freed yet). `delete p;` in `main` then frees that heap object exactly once, which is correct: there is a single `new` and a single `delete`, and the object is only accessed (via `q` or `p`) while it's still alive.

If the code had instead called `delete q;` inside the block (freeing the object early) and then used `*p` afterward, or called `delete p;` a second time after the block, that would leave a dangling pointer / cause a double-free - `p` and `q` sharing an address is fine as long as only one of them ever deletes it.